TRPL/>

KONSEP REKAYASA PERANGKAT LUNAK

(Software Engineering)

Ricak Agus Setiawan, S.T., M.SI

introduction

Kilas Balik Software Engineering

- **1950-an hingga akhir 1960-an**

  Banyaknya perdebatan tentang teknik rekayasa pada pengembangan perangkat lunak, komite sains NATO pada tahun 1968 mensponsori konferensi tentang RPL dan karena hal ini lah yang menandai awal resmi profesi rekayasa perangkat lunak

- **Akhir 1960-an sampai dengan 1980-an**

  Banyak masalah yang ditemukan para praktisi. Banyak proyek yang gagal yang dikarenakan proyek melebihi anggaran, hingga kasus kerusakan fisik dan kematian. Pada masa ini disebut sebagai Krisis perangkat lunak

- **1980-an sampai dengan sekarang**

  Banyak peneliti memfokuskan untuk menemukan teknik jitu untuk memecahkan masalah krisis perangkat lunak. Namun dengan banyaknya teknik atau metode pengembangan perangkat lunak yang muncul, tidak ada teknik pamungkas yang dapat digunakan dalam berbagai kondisi

Software Development Problem

Berangkat dari awal era evolusi computer (masa Komputer generasi ke-II) yang **sering muncul masalah** yang berhubungan dengan perangkat lunak:

- Kemampuan perangkat hardware terus meningkat hingga membuat para pengembang perangkat lunak kewalahan membangun perangkat lunak sesuai dengan perangkat keras yang ada
- Banyak pengembang tidak dapat membangun program yang cukup cepat untuk memenuhi kebutuhan bisnis dan pasar
- Meningkatnya ketergantungan masyarakat pada operasi perangkat lunak
- Kerusakan ekonomi dapat muncul bila terjadi kegagalan perangkat lunak
- Semangat para pengembang membangun perangkat lunak komputer dengan reliabilitas dan kualitas yang tinggi
- Kemampuan untuk membangun program terhambat oleh buruknya desain serta sumber daya yang tidak memadai

Software Development Problem

- Pada masa awal komputasi dalam dunia industri, pengembangan perangkat lunak masih dilihat sebagai 'Seni"
- Metode pengembangan masih sedikit dan lebih sedikit lagi orang yang memahaminya.
- Banyak pengembang yang mempelajari metode yang ada dengan cara coba-coba.
- Sering muncul pertanyaan:
  - Mengapa membutuhkan waktu yang lama untuk menyelesaikan satu program computer?
  - Mengapa biayanya sangat tinggi?
  - Dapatkah menemukan kesalahan yang ada pada program sebelum mendistribusikannya kepada pelanggan?
  - Mengapa sangat sulit untuk mengukur kemajuan pada saat perangkat lunak dikembangkan?

Software Evolution

Perkembangan perangkat lunak beriringan dengan berkembangnya perangkat keras (hardware). Beberapa **fakta tentang sejarah perangkat lunak:**

- Teori tentang software pertama kali diusulkan oleh Alan Turing pada tahun 1935 melalui esainya yang berjudul *"Nomor komputasi dengan aplikasi ke masalah Entscheidung"*
- Program komputer digital pertama yang mempunyai instruksi menggunakan kode biner muncul pada tahun 1940-an
- Kemudian bahasa pemrorgaman menjadi semakin berkembang hingga saat ini.

Software Evolution

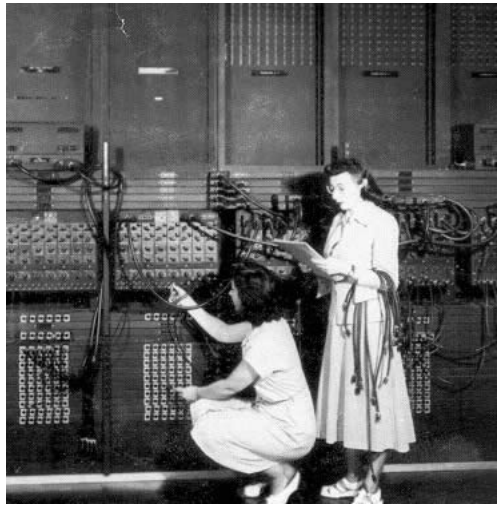
Perkembangan produk (software) yang dihasilkan terbagai menjadi beberapa masa:

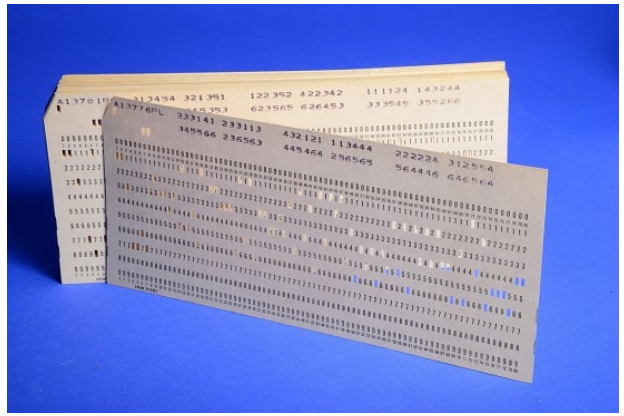
1. Era Pionir / Pemula (1950 – 1960)
2. Era Stabil (1970 – 1980)
3. Era Mikro (1980 – 1990)
4. Era Modern (1990 – 200an)

Software Evolution

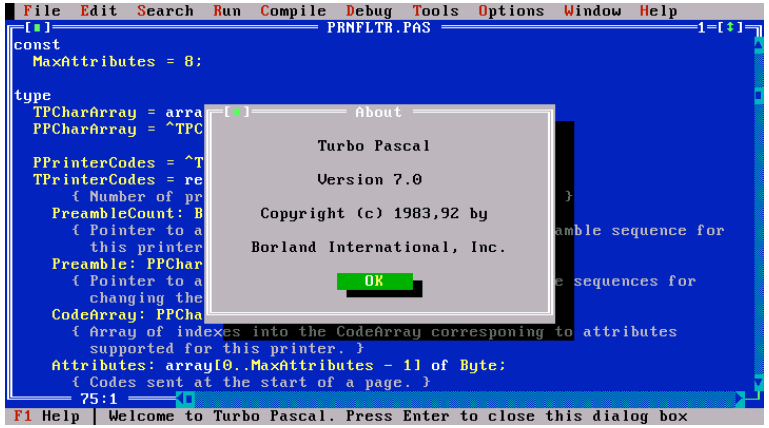
Era Pionir / Pemula (1950 – 1960)

- Bentuk perangkat lunak berupa sambungan kabel antar bagian dalam komputer atau berupa *Punched Card*
- Perangkat lunak menjadi satu kesatuan dengan perangkat kerasnya
- Satu perangkat lunak hanya menjalankan satu tujuan tertentu
- Proses dalam komputer berupa baris instruksi yang secara berurutan untuk diproses



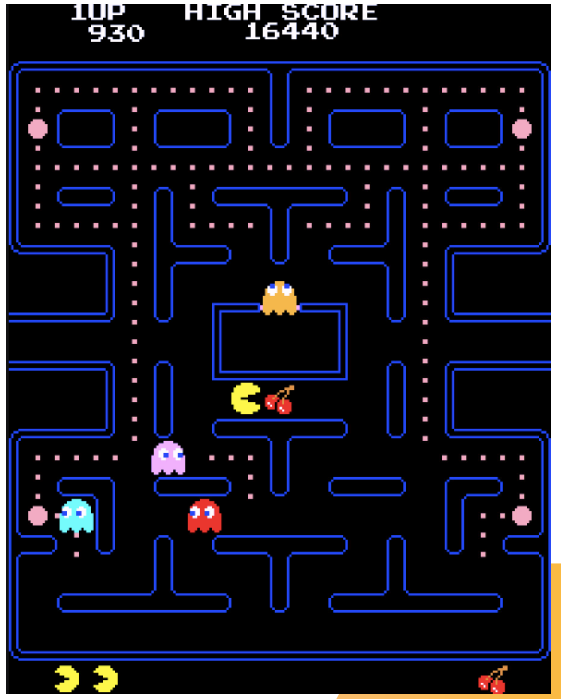


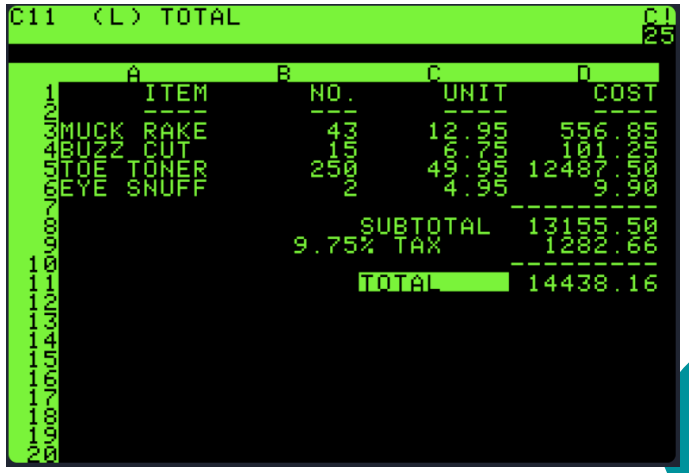
Software Evolution



Era Stabil (1970 – 1980)

- Tidak hanya digunakan kalangan peneliti dan akademi saja, namun digunakan juga dikalangan industr
- Dapat menjalankan beberapa fungsi
- Dapat membaca dan menjalankan insturksi perintah secara serentak (multitasking)
- Dapat digunakan secara multiuser dan realtime
- Mulai dikenal sistem basis data, untuk memisahkan pemroses dengan data

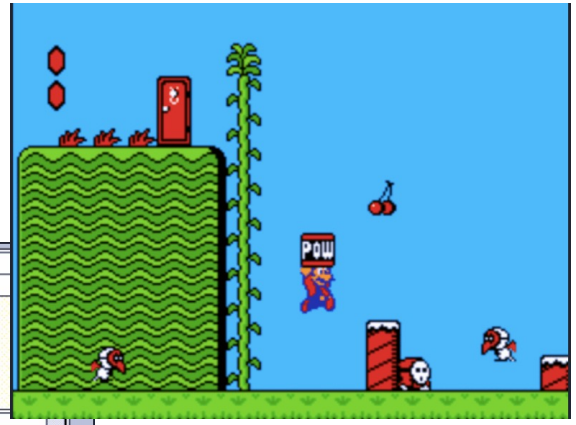


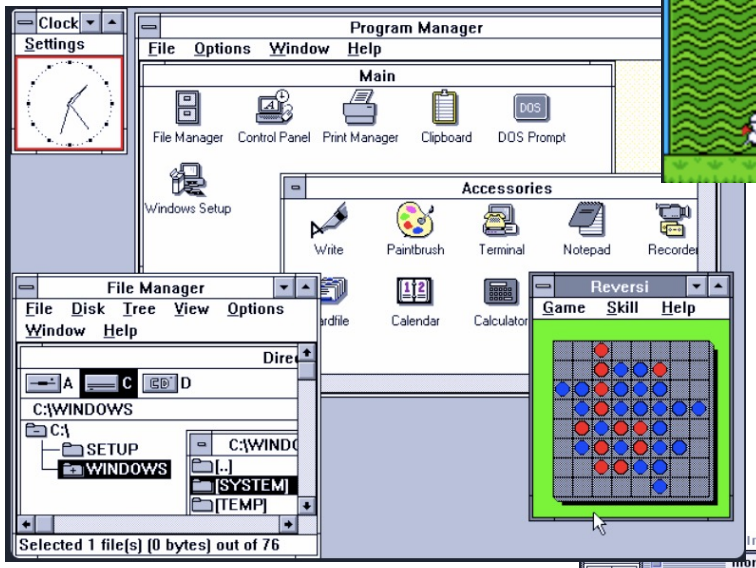


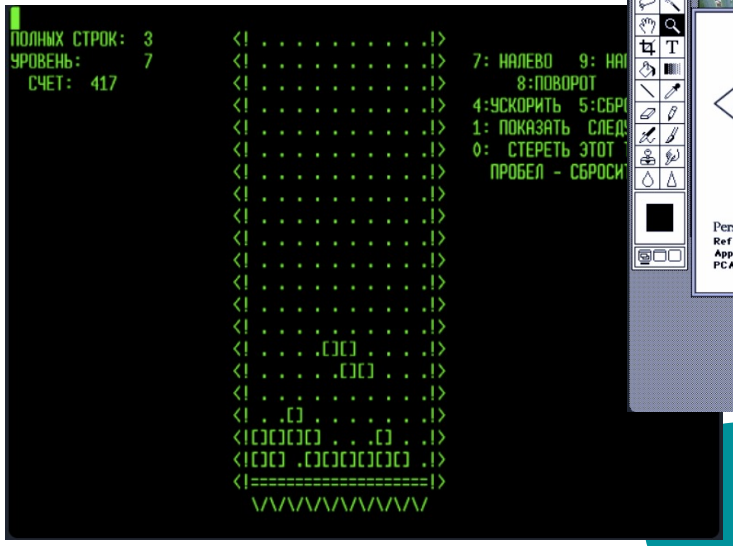
Software Evolution

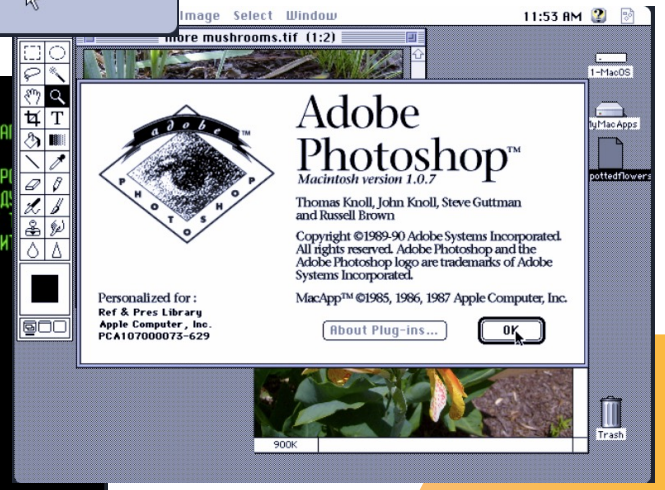
Era Mikro (1980 – 1990)

- Komputer digunakan secara perorangan dikenal dengan istilah *Personal Computer* (PC)
- Komputer dapat diinstall berbagai macam program
- Harga hardware yang rendah, menyebabkan banyak software yang dikembangkan.
- Banyak software yang diberi tambahan “intelligence”





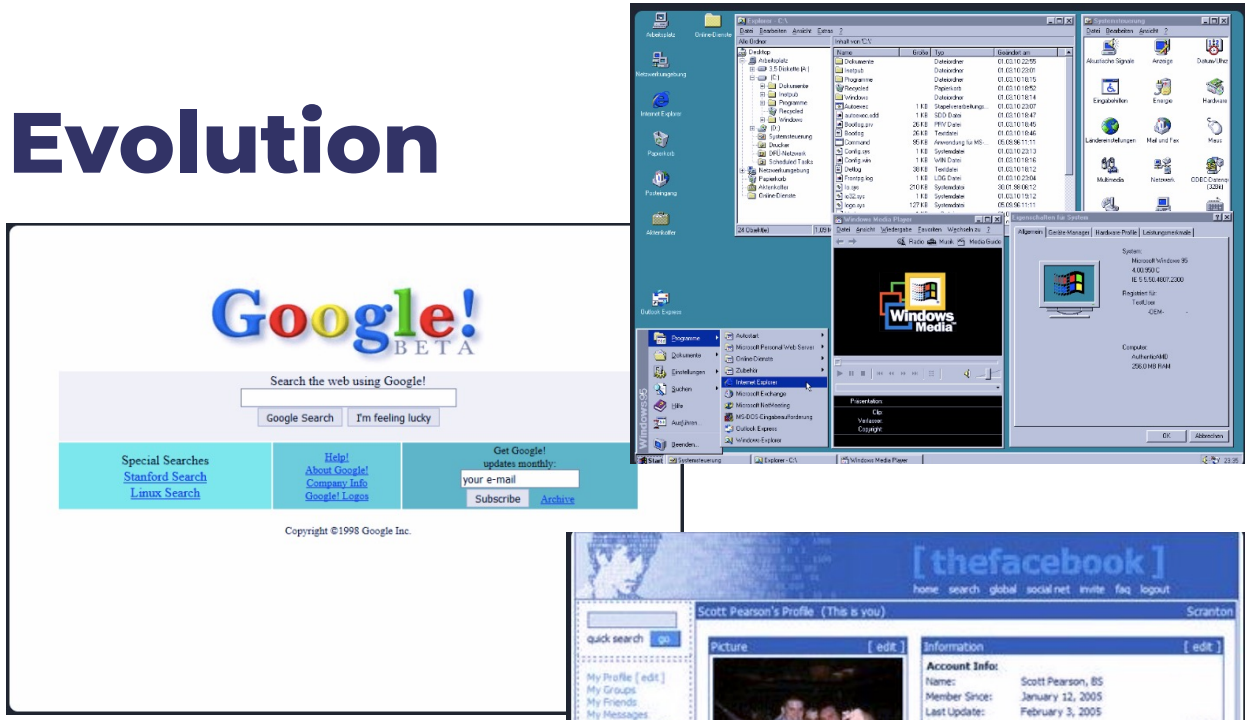




Software Evolution

Era Modern (1990 – 200an)

- Software tidak hanya berjalan di perangkat komputer, namun juga pada peralatan komunikasi, peralatan rumah tangga, dll
- Tingkat kecerdasan perangkat yang semakin meningkat (Expert System) di berbagai bidang, seperti kedokteran, meteorologi dll
- Bermunculannya A.I Machine yang dapat meniru kerja sebagian kerja otak manusia



Software Engineering

APA ITU SOFTWARE ENGINEERING?

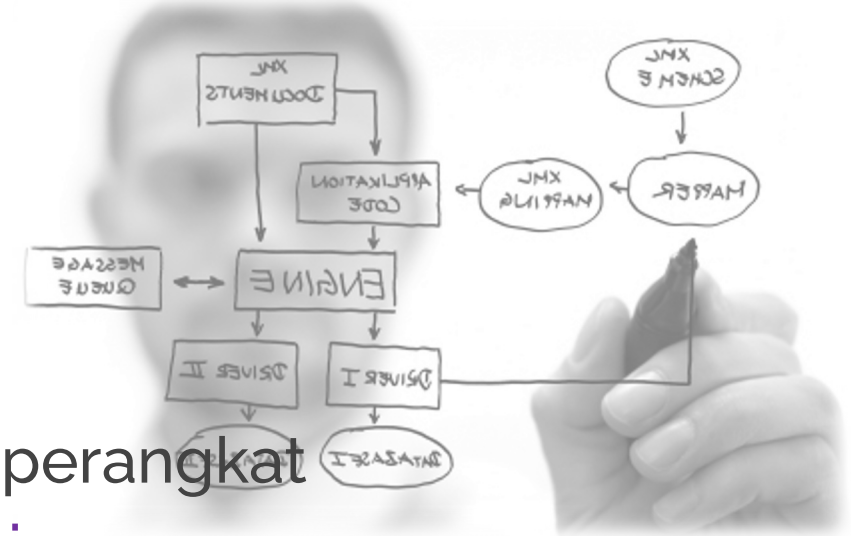
Disiplin ilmu yang membahas semua aspek produksi perangkat lunak, mulai dari tahap awal spesifikasi, desain, konstruksi, testing sampai pemeliharaan setelah digunakan *(Sommerville, 2009)*

DEFINISI (IEEE, 1993)

(1) Penerapan pendekatan sistematis, disiplin, terukur untuk pengembangan, pengoperasian, dan pemeliharaan perangkat lunak;

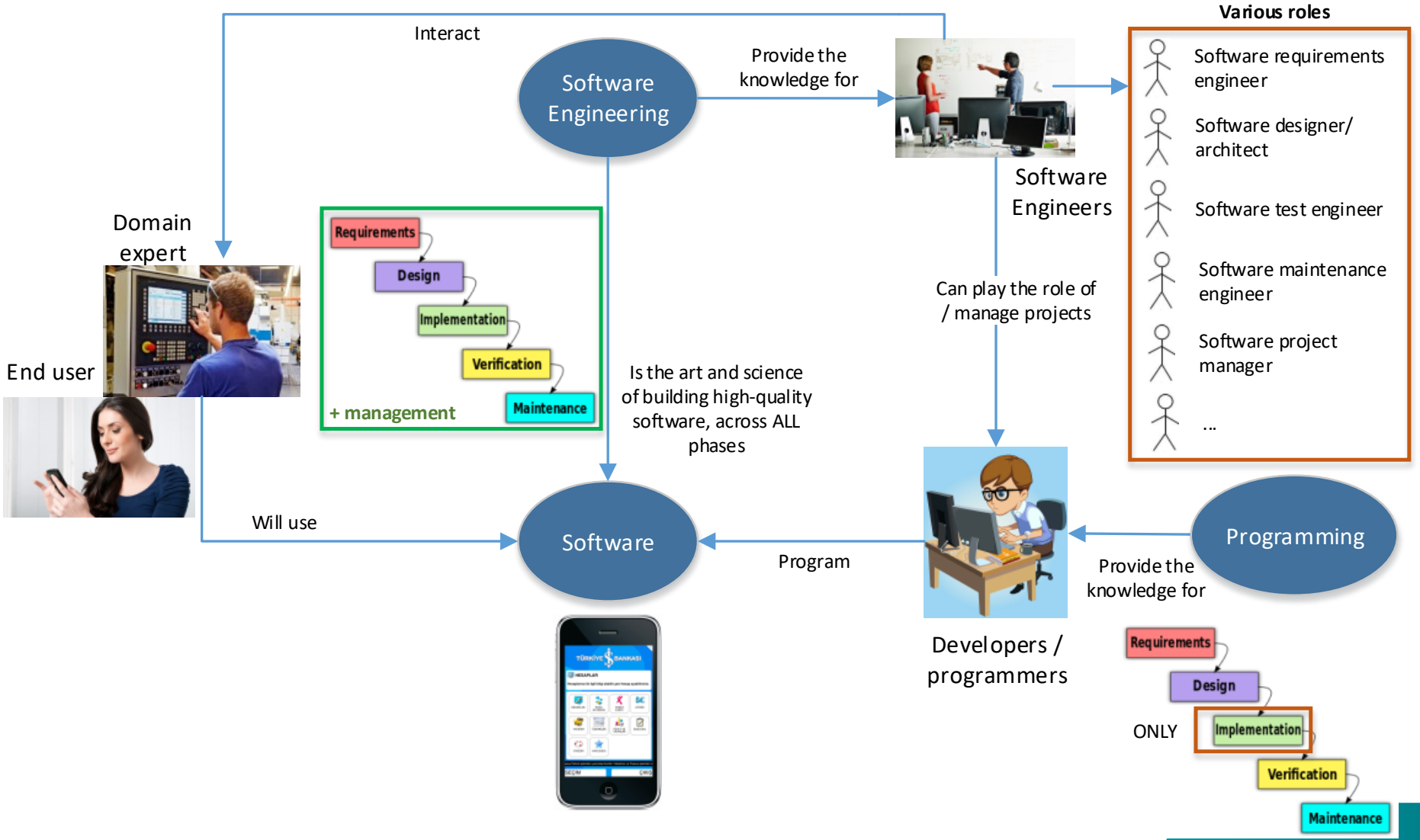
(2) Sebuah disiplin ilmu yang berkatian dengan semua aspek produksi perangkat lunak

APA ITU SOFTWARE ENGINEERING?



- **Menurut Stephen R.Schach**
  Sebuah disiplin dimana dalam menghasilkan perangkat lunak bebas dari kesalahan dan dalam pengiriman anggaran tepat waktu serta memuaskan keinginan pemakai.
- **Menurut Fritz Bauer**
  Penetapan dan penggunaan prinsip rekayasa dalam rangka memperoleh perangkat lunak yang dapat dipercaya dan dapat bekerja secara efisien pada mesin nyata.

APA ITU SOFTWARE ENGINEERING?



APA ITU SOFTWARE ENGINEERING?

- Rekayasa Perangkat Lunak bukan sekedar bahasa pemrograman namun untuk menjadi software engineer (insinyur perangkat lunak) yang baik salah satunya harus mahir dalam bahasa pemrograman.
- Rekayasa perangkat lunak adalah keilmuan dan penerapan rekayasa untuk desain, pengembangan, pengujian dan pemeliharaan sistem perangkat lunak

Software Engineer VS Software Developer

Tanggung Jawab:

- Riset, desain, mengembangkan, menguji, dan maintenance
- Merancang software sesuai kebutuhan user
- Menjalankan konsep SDLC

Kemampuan yang harus dimiliki:

- Familiar atau menguasai bahasa programming
- Menggunakan tools pengujian software
- Analisis dan desain software
- Kerjasama dan Komunikasi yang baik

Tanggung Jawab:

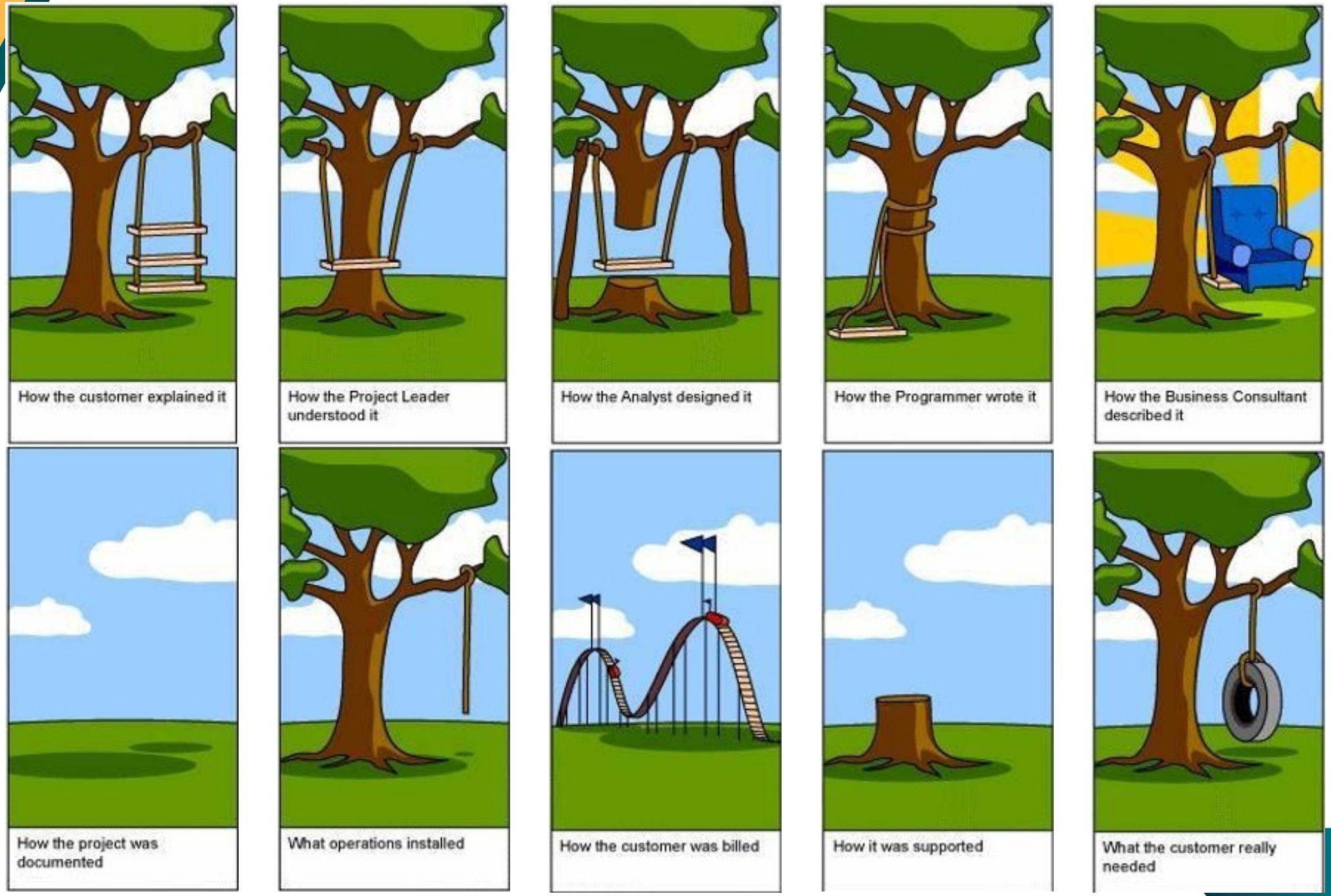
- Membangun software sebaik-baiknya
- Menguji efisiensi dan fungsionalitas software
- Melakukan maintenance untuk keamanan dan ketahanan

Kemampuan yang harus dimiliki:

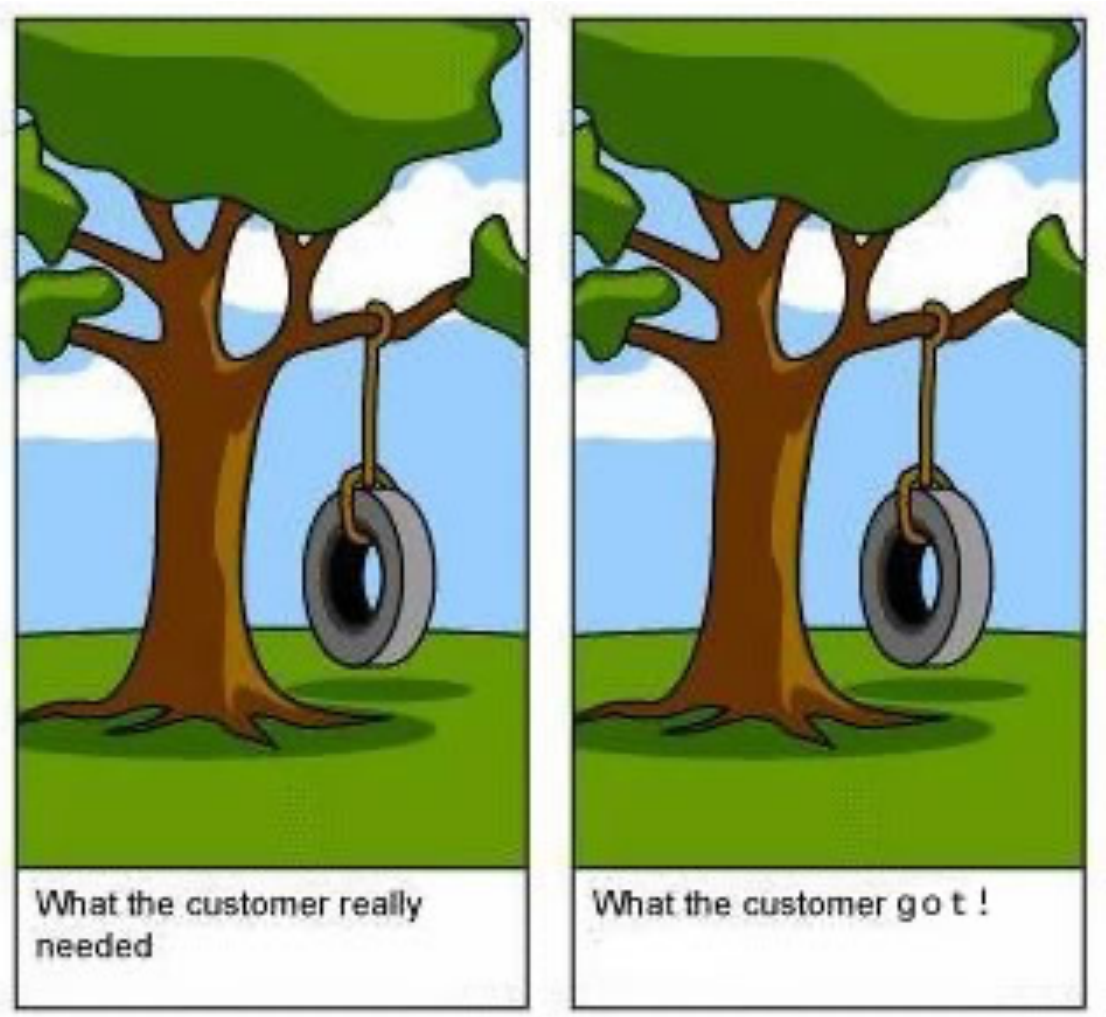
- Familiar atau menguasai bahasa programming
- Paham logika kode bahasa pemrograman yang belum dikuasai
- Kerjasama dan Komunikasi yang baik

TANTANGAN REKAYASA PERANGKAT LUNAK

- Bagaimana dapat mengembangkan perangkat lunak dengan waktu singkat, biaya rendah dengan kualitas tinggi?
- Bagaimana dapat menguji perangkat lunak dengan waktu singkat, biaya rendah dengan kualitas tinggi?
- Ketika kita memiliki tim programmer yang besar, bagaimana kita dapat menugaskan orang-orang terbaik untuk tugas pengembangan dan pengujian?
- Bagaimana memastikan bahwa telah menanyakan persyaratan perangkat lunak dari klien dengan cara yang paling efisien dan efektif?


How the customer explained it
How the Project Leader understood it
How the Analyst designed it
How the Programmer wrote it
How the Business Consultant described it
How the project was documented
What operations installed
How the customer was billed
How it was supported
What the customer really needed

TUJUAN REKAYASA PERANGKAT LUNAK

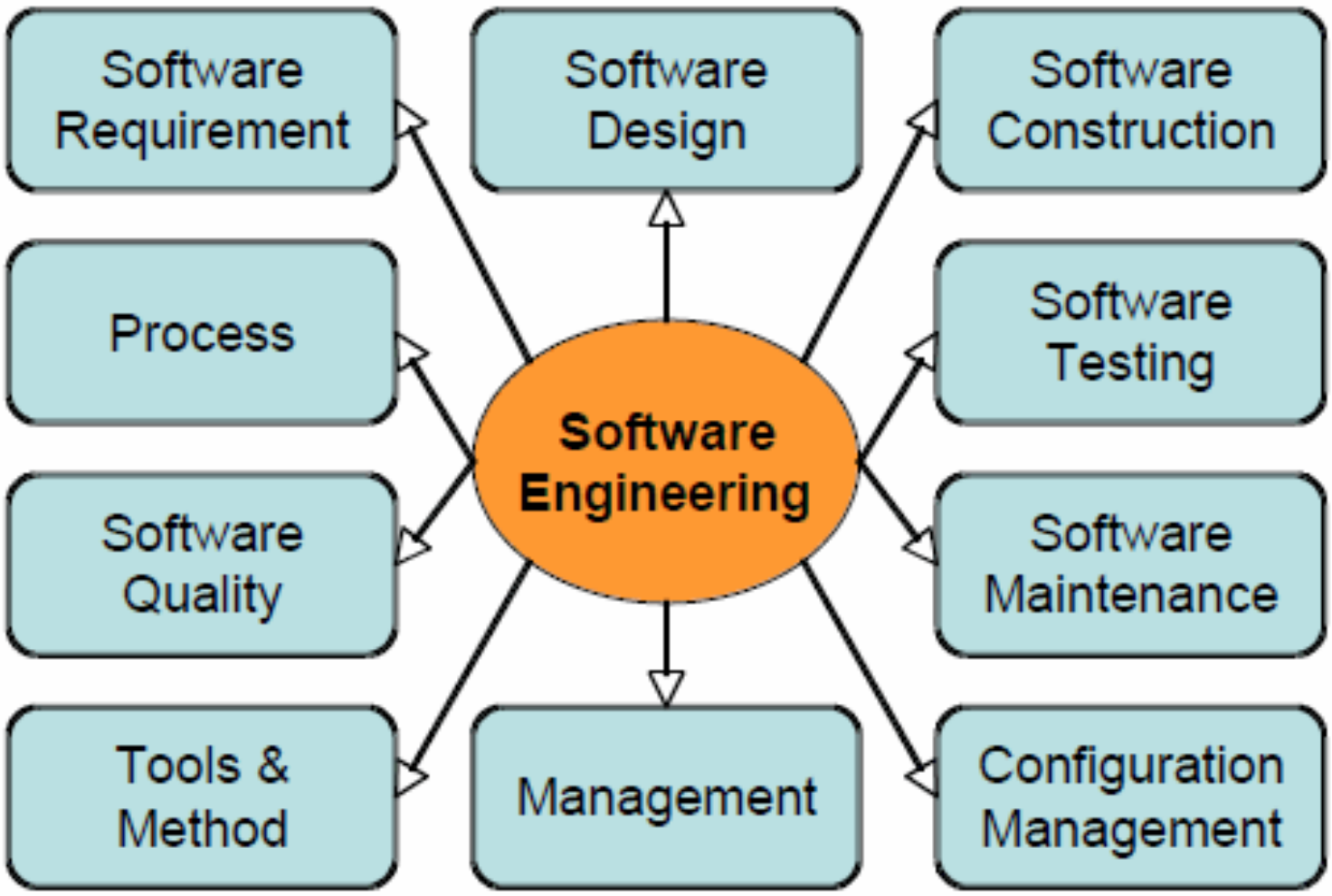


Perangkat Lunak yang baik:

- Harus memberikan fungsi dan kinerja (performa) yang dibutuhkan pengguna perangkat lunak
- Harus dapat dipelihara, dapat diandalkan, dan dapat digunakan

(Sommerville, 2012)

RUANG LINGKUP REKAYASA PERANGKAT LUNAK



RUANG LINGKUP PERANGKAT LUNAK

1. Software requirements berhubungan dengan spesifikasi kebutuhan dan persyaratan perangkat lunak.
2. Software design mencakup proses penentuan arsitektur, komponen, antarmuka, dan karakteristik lain dari perangkat lunak.
3. Software construction berhubungan dengan detil pengembangan perangkat lunak, termasuk algoritma, pengkodean, pengujian, dan pencarian kesalahan.
4. Software testing meliputi pengujian pada keseluruhan perilaku perangkat lunak.
5. Software maintenance mencakup upaya-upaya perawatan ketika perangkat lunak telah dioperasikan.

RUANG LINGKUP PERANGKAT LUNAK

6. Software configuration management berhubungan dengan usaha perubahan konfigurasi perangkat lunak untuk memenuhi kebutuhan tertentu.
7. Software engineering management berkaitan dengan pengelolaan dan pengukuran RPL, termasuk perencanaan proyek perangkat lunak.
8. Software engineering tools and methods mencakup kajian teoritis tentang alat bantu dan metode RPL.
9. Software engineering process berhubungan dengan definisi, implementasi, pengukuran, pengelolaan, perubahan dan perbaikan proses RPL.
10. Software quality menitikberatkan pada kualitas dan daur hidup perangkat lunak.

SOFTWARE MYTHS

MANAGEMENT

Mitos yang beredar dari sudut pandang manajamen

CUSTOMER

Mitos yang beredar dari sudut pandang pelanggan / user

PRACTITIONER'S

Mitos yang beredar dari sudut pandang perkerja tekniks / programmer

MITOS SOFTWARE SUDUT PANDANG MANAJEMEN

MANAGEMENT MYTHS

Mitos 1:

"Tim kita sudah memiliki buku yang penuh dengan standar dan prosedur untuk membangun perangkat lunak, bukankah itu memberi orang-orang saya semua yang perlu mereka ketahui?"

Realita:

- Apakah para praktisi *software* sadar akan keberadaan standar?
- Apakah standar itu mencerminkan praktik rekayasa perangkat lunak modern?
- Apakah standar itu lengkap? Apakah efisien untuk meningkatkan waktu pengiriman dan sambil tetap mempertahankan fokus kualitas?

MANAGEMENT MYTHS

Mitos 2:

"Jika kita terlambat menyelesaikan proyek software, kita dapat menambahkan lebih banyak programmer untuk mengejar keterlambatan"

Realita:

- Perangkat Lunak bukan melalui proses manufaktur. Menambah orang ke proyek akan membuat *software* tertahan.
- Orang dapat tambahkan tetapi dengan cara terencana dan terkoordinasi dengan baik.

MANAGEMENT MYTHS

Mitos 3:

"Jika kita memutuskan untuk mengalihkan proyek software ke pihak ketiga, maka saya bisa santai dan membiarkan mereka membangunnya"

Realita:

Jika sebuah organisasi tidak memahami bagaimana mengelola dan mengontrol proyek *software* secara internal, organisasi akan selalu kesulitan ketika mengalihkan proyek perangkat lunak.

MITOS SOFTWARE SUDUT PANDANG PELANGGAN

CUSTOMER MYTHS

Mitos 1:

"Pernyataan umum sudah cukup untuk mulai membuat program, kita bisa melengkapi detailnya nanti"

Realita:

- Pendefinisian awal yang buruk adalah penyebab utama kegagalan proyek *software*.
- Sebuah penjelasan formal dan rinci mulai dari domain informasi, fungsi, perilaku, kinerja, antarmuka hingga kriteria validasi merupakan hal yang sangat penting
- Karakteristik ini dapat ditentukan hanya setelah komunikasi menyeluruh antara pelanggan dan pengembang.

CUSTOMER MYTHS

Mitos 2:

"Persyaratan proyek dapat terus berubah, tetapi dapat dengan mudah diakomodasi karena software bersifat fleksibel"

Realita:

- Pelanggan dapat meninjau persyaratan dan merekomendasikan modifikasi dengan dampak biaya yang relatif kecil.
- Ketika perubahan diminta selama desain perangkat lunak, dampak biaya tumbuh dengan cepat

MITOS SOFTWARE SUDUT PANDANG PRAKTISI PERANGAKT LUNAK

PRACTITIONER'S MYTHS

Mitos 1:

"Sekali kami menulis program dan menjalankannya maka pekerjaan kami selesai"

Realita:

- Terdapat istilah "semakin cepat Anda mulai menulis kode, semakin lama waktu yang Anda perlukan untuk menyelesaikannya"
- Karena data industri menunjukkan bahwa sekitar 60%-80% upaya (pengembangan / perbaikan) untuk *software* baru akan muncul setelah *software* dikirimkan ke pelanggan pertama kalinya

PRACTITIONER'S MYTHS

Mitos 2:

"Sampai saya mendapatkan program "running", saya tidak memiliki cara untuk menilai kualitasnya"

Realita:

Salah satu mekanisme jaminan kualitas perangkat lunak yang paling efektif dapat diterapkan dari awal proyek

PRACTITIONER'S MYTHS

Mitos 3:

"Satu-satunya produk yang dapat dikirimkan untuk proyek yang sukses adalah program bekerja"

Realita:

- Program yang bekerja hanyalah satu bagian dari konfigurasi perangkat lunak yang mencakup banyak elemen.
- Dokumentasi memberikan dasar untuk rekayasa yang sukses dan, yang lebih penting, panduan untuk dukungan perangkat lunak

PRACTITIONER'S MYTHS

Mitos 4:

"Rekayasa perangkat lunak akan mengharuskan kita membuat dokumentasi yang banyak yang tidak perlu dan akan selalu memperlambat kita"

Realita:

- Rekayasa perangkat lunak bukan tentang membuat dokumen. Ini tentang menciptakan kualitas.
- Kualitas yang lebih baik mengarah pada pengurangan pengerjaan ulang. karena pengurangan pengerjaan ulang menghasilkan waktu pengiriman yang lebih cepat

Permasalahan Yang Sering Terjadi Saat Pengembangan Software

- Konsep atau blueprint pengembagan yang tidak matang
- Komunikasi vendor dan klien kurang in-line
- Overpromise dan Undedeliver
- Penambahan fitur di luar kontrak kerja
- Proyek berhenti di tengah jalan
- Komunikasi internal tim yang kurang professional
- Vendor menghilang di tengah proyek
- Tim kekurangan SDM yang berpengalaman
- Tim tidak mampu tapi mengiyakan
- Masalah manajemen keuangan tim